Micrometer 1.17 · Spring Boot 4.1

# Micrometer: mide tu app

De logs con milisegundos a métricas que puedes consultar, graficar y alertar.



Chamba Lab · 2026

# **Un registro, cuatro medidores:** todo pasa por MeterRegistry

**Counter**
`pedidos.increment()`

**Timer**
`checkout.record(...)`

**Meter Registry**

**Gauge**
`registry.gauge(...)`

**DistributionSummary**
`montos.record(129.90)`

Tu código habla con el registro; el registro habla con Prometheus, Datadog o el que elijas.

# **Medir a mano:** un log por cada checkout

Cada línea es un dato suelto: para saber el p95 de ayer hay que hacer grep y promediar a mano.
Y si el cobro lanza una excepción, ese checkout ni siquiera queda registrado.

```
public Pedido confirmar(Carrito carrito) {
  long inicio = System.currentTimeMillis();

  Pedido pedido = pagos.cobrar(carrito);

  long ms = System.currentTimeMillis() - inicio;
  log.info("checkout tardó {} ms", ms);
  return pedido;
}

// checkout tardó 212 ms
// checkout tardó 1840 ms
// checkout tardó 97 ms  ...  y 40 000 más
```



# Mide una vez, publica donde quieras

## Micrometer es a las métricas lo que SLF4J a los logs.

- El mismo código sirve para Prometheus, Datadog o CloudWatch
- Promedios y p95 se calculan en el servidor, no con grep
- Spring Boot ya mide HTTP, JVM y base de datos por ti

# **Dos dependencias:** Spring Boot hace el resto

pom.xml

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!— el backend: cambia solo esta línea —>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Actuator crea el `MeterRegistry` y lo inyecta donde lo pidas. La versión la fija el BOM de Spring Boot.

El endpoint viene cerrado. Ábrelo en `application.properties`: `management.endpoints.web.exposure.include=health,prometheus`

Prometheus lee `/actuator/prometheus` cada pocos segundos.

# **Timer:** cuánto tarda y cuántas veces pasa



**Timer:** cada duración cae en su cubeta

- Un solo medidor da conteo, suma y máximo
- El histograma permite calcular p95 sumando instancias

```
Timer checkout = Timer
    .builder("tienda.checkout")
    .description("confirmar un pedido")
    .publishPercentileHistogram()
    .register(registry);

Pedido confirmar(Carrito carrito) {
  return checkout.record(
      () → pagos.cobrar(carrito));
}
```

Mide también cuando el cobro lanza una excepción.

# **Counter y Gauge:** lo que ocurre y lo que hay

```
Counter pedidos = Counter
    .builder("tienda.pedidos")
    .register(registry);
pedidos.increment();

AtomicInteger carritos = registry.gauge(
    "tienda.carritos.activos",
    new AtomicInteger(0));
carritos.incrementAndGet(); // entra
carritos.decrementAndGet(); // sale
```

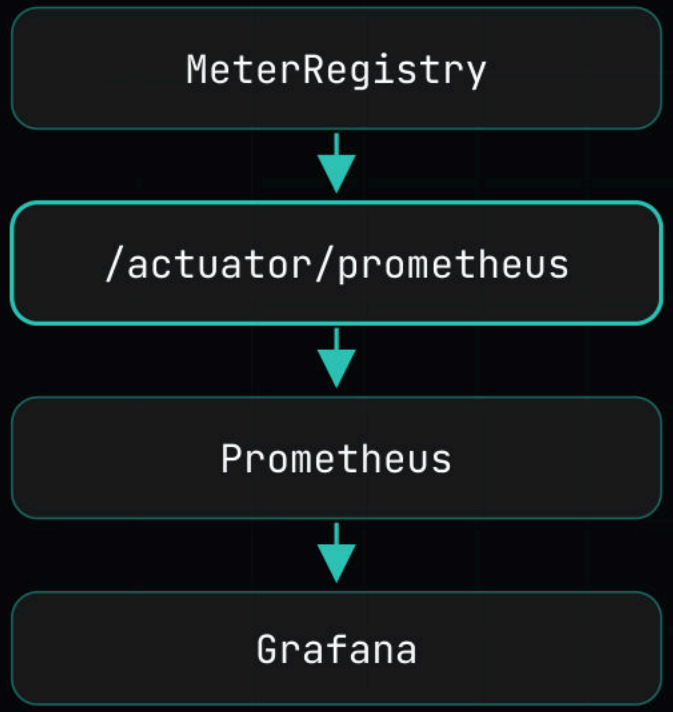
Guarda esta referencia: el gauge solo tiene una débil. Si el GC la recoge, reporta NaN.



un clic por evento · el nivel del frasco

- Counter solo sube
- Gauge sube y baja: se lee en el momento

# **Del código a la gráfica:** lo que lee Prometheus




```
# /actuator/prometheus (extracto)
tienda_checkout_seconds_count 1342.0
tienda_checkout_seconds_sum 402.6
tienda_checkout_seconds_max 1.9

tienda_pedidos_total 1298.0
tienda_carritos_activos 37.0
```

sum ÷ count = promedio. Con el histograma se suma la serie _bucket, de donde sale el p95.

- Los puntos pasan a guiones bajos y se añade la unidad

# La trampa de las etiquetas: cada valor distinto es una serie nueva

## Pocos valores posibles

```
registry.counter("tienda.pedidos",
    "metodo", carrito.metodo())
  .increment();

// tarjeta · yape · efectivo
// = 3 series
```

Filtra y agrupa sin costo: "¿cuántos pedidos con Yape?"

## Un valor por cliente

```
registry.counter("tienda.pedidos",
    "cliente", cliente.id())
  .increment();

// c-00001 · c-00002 · … · c-48210
// = 48 210 series y subiendo
```

La memoria de la app y de Prometheus crece sin límite

# **Observation API:** una medición, métrica y traza




- Low cardinality: va a métricas y trazas
- High cardinality: solo a trazas
- Spring inyecta el ObservationRegistry

```
Pedido confirmar(Carrito carrito) {
  return Observation
    .createNotStarted(
        "tienda.checkout", observations)
    .lowCardinalityKeyValue(
        "metodo", carrito.metodo())
    .highCardinalityKeyValue(
        "pedido.id", carrito.id())
    .observe(
        () → pagos.cobrar(carrito));
}
```

El id por pedido va aquí: nunca llega a la métrica, así que no la infla.

# **Matriz de decisión:** qué medidor para cada pregunta

| | Counter | Timer | Gauge | Observation |
|---|---|---|---|---|
| **Responde** | ¿Cuántas veces pasó? | ¿Cuánto tarda y cuántas veces? | ¿Cuánto hay ahora? | Lo del Timer + la traza |
| **En la tienda** | Pedidos confirmados | Duración del checkout | Carritos activos | Checkout con su id de pedido |
| **Trampa** | Contar lo que se puede medir con un Timer | Percentiles locales: no se suman entre instancias | Perder la referencia: NaN | Un id como low cardinality |

¿Dura? Timer, u Observation si también quieres trazas. ¿Ocurre? Counter. ¿Es un nivel en este momento? Gauge.

# Lo que te llevas

- Timer para lo que dura, Counter para lo que ocurre, Gauge para lo que hay
- Etiquetas con pocos valores; los ids van a las trazas
- Instrumenta contra MeterRegistry: el backend es solo una dependencia

```
> docs: docs.micrometer.io/micrometer/reference
> spring: docs.spring.io/spring-boot/reference/actuator/metrics.html
> repo: github.com/Chamba-Lab
```